

Informe Individual

Análisis de Sistemas Distribuidos

Apache Cassandra: Transparencias, CAP,
Comunicación y Tolerancia a Fallos

Cristhian Daniel Pacheco Cárdenas

Mayo 2026

Índice

1. Introducción	3
2. Transparencias de Distribución	3
2.1. ¿Qué son las transparencias?	3
2.2. Transparencia de acceso	3
2.3. Transparencia de localización	4
2.4. Transparencia de replicación	4
2.5. Transparencia de fallos	4
2.6. Transparencia de concurrencia	4
2.7. Resumen de transparencias	5
3. Modelo CAP	5
3.1. ¿Qué plantea el teorema CAP?	5
3.2. Cassandra como sistema AP	5
4. Modelos de Comunicación	6
4.1. Los dos modelos clásicos	6
4.2. El protocolo Gossip en Cassandra	6
5. Tolerancia a Fallos	6
5.1. Replicación	7
5.2. Detección de fallos adaptativa	7
6. Análisis Comparativo	7
7. Conclusiones	8

1. Introducción

En 2008, Facebook necesitaba que sus usuarios pudieran buscar mensajes dentro de su bandeja de entrada. Cientos de millones de usuarios, Miles de millones de mensajes al día, y la expectativa de que el sistema nunca se interrumpiera. Ninguna base de datos tradicional podía cumplir con esos requisitos, por lo que Facebook construyó su propia solución y esta fue Apache Cassandra, diseñada para funcionar sobre cientos de servidores baratos sin depender de ningún punto central que, si fallaba, derribara todo lo demás [4].

Un sistema distribuido es un conjunto de computadoras independientes que aparece a sus usuarios como un único sistema coherente [5]. Muchas máquinas trabajando juntas, pero percibidas como una sola.

2. Transparencias de Distribución

2.1. ¿Qué son las transparencias?

Cuando se usa un sistema distribuido, el usuario no debería notar que hay múltiples máquinas detrás. Si el sistema logra ocultar esa complejidad, el sistema es transparente. Existen varios tipos de transparencia que un correcto sistema distribuido debería ofrecer [5]. Las más importantes en este análisis son cinco: acceso, localización, replicación, fallos y concurrencia.

2.2. Transparencia de acceso

Esta transparencia consiste en que el usuario puede interactuar con el sistema de la misma forma sin importar cómo estén organizados internamente los datos [5]. En Cassandra, el cliente escribe sus consultas en CQL (*Cassandra Query Language*), un lenguaje similar a SQL. No necesita saber en qué servidor está guardado cada dato ni cómo está estructurado el clúster por dentro.

Esta transparencia está completamente implementada en Cassandra. El sistema hace un buen trabajo ocultando la complejidad de acceso al cliente.

2.3. Transparencia de localización

El cliente no necesita saber dónde están físicamente sus datos. Cassandra organiza sus nodos en un esquema circular llamado “anillo” y distribuye los datos automáticamente. Cuando llega una solicitud, cualquier nodo del clúster puede recibirla y redirigirla al nodo correcto sin que el cliente lo note [4].

Esta transparencia también está completamente implementada. El usuario interactúa con Cassandra como si fuera una sola base de datos, aunque sus datos estén repartidos por decenas de servidores en distintos países.

2.4. Transparencia de replicación

Cassandra guarda copias de cada dato en varios nodos al mismo tiempo. Si un servidor falla, los datos siguen disponibles en los demás. El cliente no tiene que gestionar esas copias: el sistema lo hace solo [4].

Sin embargo, como Cassandra prioriza la disponibilidad sobre la consistencia, puede ocurrir que dos nodos tengan versiones ligeramente distintas del mismo dato durante un breve período. El cliente no recibe ningún aviso de esto.

Esta transparencia es parcial, debido a que la replicación en sí es invisible, pero sus consecuencias (datos temporalmente distintos entre réplicas) sí pueden afectar al usuario sin que él lo sepa.

2.5. Transparencia de fallos

El sistema debería continuar funcionando aunque algunos componentes fallen, sin que el usuario lo note, enmascarar los fallos es un problema difícil en sistemas distribuidos, y es incluso imposible bajo ciertas condiciones realistas” [5].

Cassandra maneja bien los fallos comunes: si un servidor se apaga, el sistema continúa respondiendo. Pero ante una separación severa de la red que divida el clúster en dos grupos incomunicados, el sistema puede devolver datos desactualizados sin avisar al cliente que algo está mal.

Esta transparencia también es parcial debido a que funciona en condiciones normales, pero ante fallos de red graves el usuario puede recibir información incorrecta sin que él lo sepa.

2.6. Transparencia de concurrencia

Cuando muchos usuarios modifican el mismo dato al mismo tiempo, el sistema debería manejar esos conflictos de forma coherente [5]. Cassandra resuelve los conflictos de escritura haciendo que gane la escritura más reciente según su marca de tiempo. El problema es que si dos escrituras ocurren casi al mismo tiempo en distintos nodos, una de ellas puede perderse sin que el sistema lo reporte.

Esta transparencia también es parcial, debido a que funciona en escenarios de baja concurrencia, pero en alta concurrencia sobre el mismo dato puede haber pérdida silenciosa de información.

2.7. Resumen de transparencias

De las cinco transparencias analizadas, Cassandra implementa de forma completa las de acceso y localización, que son las más visibles para el usuario. Las de replicación, fallos y concurrencia son parciales porque el sistema prioriza seguir funcionando sobre el hecho de que todos los datos sean siempre exactos en todo momento.

Cassandra sacrifica exactitud inmediata para ganar disponibilidad y escala. Esta decisión tiene sentido para redes sociales o sistemas de registro de eventos, pero puede ser problemática en aplicaciones donde la exactitud de los datos en todo momento es crucial, como sistemas bancarios o de salud. El mayor riesgo es que el desarrollador use Cassandra sin conocer estas limitaciones, asumiendo que el sistema garantiza consistencia fuerte cuando en realidad no lo hace.

3. Modelo CAP

3.1. ¿Qué plantea el teorema CAP?

El teorema CAP, formulado por Eric Brewer, establece que un sistema distribuido no puede garantizar al mismo tiempo las tres propiedades siguientes: [1]:

- **Consistencia (C):** Todos los nodos ven siempre la misma versión del dato.
- **Disponibilidad (A):** El sistema siempre responde, aunque algún componente haya fallado.
- **Tolerancia a particiones (P):** El sistema sigue funcionando aunque haya una falla de comunicación entre sus nodos.

Cuando la red falla y dos grupos de nodos no pueden comunicarse, el sistema debe elegir entre seguir respondiendo con los datos que tiene aunque puedan estar desactualizados, o dejar de responder hasta que la situación se resuelva para asegurar que los datos sean correctos. Esa es la elección fundamental [1].

3.2. Cassandra como sistema AP

Cassandra elige disponibilidad y tolerancia a particiones, sacrificando consistencia fuerte. Cuando hay una falla de red, sigue respondiendo a las solicitudes aunque distintos nodos puedan tener versiones ligeramente distintas del mismo dato. La reconciliación ocurre después, en segundo plano cuando la red se recupera [4].

Comparado con sistemas CP como Google Spanner o HBase, Cassandra ofrece mayor velocidad y disponibilidad continua, pero a cambio de garantías más débiles sobre la

exactitud de los datos en todo momento. Google Spanner garantiza que todos los nodos siempre tienen la misma versión, pero para lograrlo necesita coordinación global que introduce latencia y reduce la disponibilidad ante particiones.

4. Modelos de Comunicación

4.1. Los dos modelos clásicos

Los dos modelos clásicos más conocidos son RPC y MOM

El primero es **RPC** (*Remote Procedure Call*), también conocido como Llamada a procedimiento remoto: Funciona de manera sincrónica. El cliente hace una solicitud y espera, detenido, hasta que recibe la respuesta. Es simple de programar y fácil de entender, pero si el servidor tarda mucho o falla, el cliente queda esperando sin poder hacer nada más.

El segundo es **MOM** (*Message-Oriented Middleware*), también conocido como Middleware orientado a mensajes: Funciona de manera asíncrona. El cliente envía el mensaje y continúa trabajando sin esperar. El servidor procesa cuando puede y responde de la misma forma. Es más robusto ante fallos y adecuado para operaciones largas, pero más complejo de implementar [3].

El modelo RPC es conveniente cuando la respuesta es rápida y previsible; mientras que el modelo MOM es mejor usar cuando la operación puede tardar o cuando el sistema debe resistir que el receptor no esté disponible

4.2. El protocolo Gossip en Cassandra

Cassandra no usa ni RPC ni MOM para la comunicación interna entre sus nodos. Usa el **protocolo Gossip**, que funciona de forma completamente diferente [4].

Cada nodo, de forma periódica, elige de manera aleatoria a otro nodo del clúster e intercambia con él información sobre el estado del sistema. Esa información se propaga de nodo en nodo rápidamente y sin necesitar un servidor central que coordine todo.

Gossip elimina cualquier punto central de fallo que tendría un servidor de coordinación, y escala bien a medida que se agregan nodos. Sin embargo, su principal limitación es que la información tarda un tiempo en propagarse por todo el clúster: en sistemas muy grandes o con muchos cambios simultáneos, puede haber momentos en que distintos nodos tengan visiones ligeramente distintas del estado del sistema.

5. Tolerancia a Fallos

Cassandra fue diseñada para tener tolerancia a fallos, en un clúster de cientos de servidores, siempre habrá alguno fallando en cualquier momento dado [4].

Las técnicas de tolerancia a fallos se clasifican en dos grupos [2]: las reactivas, que

actúan después de que el fallo ya ocurrió (reintentos, replicación, checkpointing), y las proactivas, que anticipan el fallo antes de que suceda y reemplazan componentes mientras aún funcionan (migración preventiva, auto-recuperación).

5.1. Replicación

La estrategia más importante de Cassandra es guardar copias de cada dato en varios nodos al mismo tiempo. Si un servidor falla, los demás tienen las mismas copias y pueden responder sin interrupciones. El número de copias es configurable por el administrador [4]. Estas técnicas no evitan que el nodo falle, pero asegura que el fallo no afecte al cliente.

5.2. Detección de fallos adaptativa

Cassandra no decide de forma binaria si un nodo está vivo o muerto. Calcula continuamente una puntuación de probabilidad de fallo para cada nodo, basada en el historial de sus respuestas [4]. Si un nodo empieza a responder lentamente, la puntuación sube de forma gradual. Esto evita declarar muerto a un nodo que simplemente está bajo carga temporal, reduciendo falsas alarmas.

6. Análisis Comparativo

Los cuatro ejes no son independientes entre sí: forman un sistema coherente de compromisos que se refuerzan mutuamente.

La elección AP en el modelo CAP explica directamente por qué las transparencias de replicación, fallos y concurrencia son solo parciales: el sistema acepta devolver datos que pueden no ser perfectamente actuales a cambio de seguir funcionando siempre. El protocolo Gossip refuerza esa arquitectura porque permite que los nodos se coordinen sin un punto central que podría convertirse en un cuello de botella o en un punto único de fallo. Y los mecanismos de tolerancia a fallos garantizan que esa disponibilidad prometida se mantenga en la práctica incluso cuando partes del sistema fallen.

Comparado con un sistema CP como Google Spanner, Cassandra es más rápida y más disponible, pero menos precisa. Spanner garantiza que todos los nodos siempre tienen la misma versión de los datos, pero para lograrlo introduce latencia de coordinación global y puede dejar de responder ante particiones de red, algo que Cassandra nunca hace.

El mayor riesgo en el uso de Cassandra no es técnico sino el hecho usar el sistema sin comprender sus compromisos. Un desarrollador que asuma que Cassandra garantiza consistencia fuerte tomará decisiones de diseño incorrectas que solo se manifestarán como problemas en producción, bajo condiciones de fallo o alta concurrencia.

7. Conclusiones

Apache Cassandra es un sistema distribuido bien diseñado para su propósito original. Sus decisiones forman un conjunto coherente:

Las transparencias de acceso y localización están completamente implementadas, lo que simplifica el trabajo del desarrollador. Las de replicación, fallos y concurrencia son parciales como consecuencia directa de priorizar la disponibilidad sobre la consistencia perfecta.

En conclusión, Cassandra demuestra que en sistemas distribuidos no existen soluciones perfectas sino que toda decisión de diseño implica un compromiso. El valor del análisis crítico está en entender exactamente qué se está ganando y qué se está cediendo con cada elección, para poder aplicar la tecnología correcta al problema correcto.

Referencias

- [1] E. A. Brewer, “CAP Twelve Years Later: How the ‘Rules’ Have Changed,” *Computer*, vol. 45, no. 2, pp. 23–29, Feb. 2012, doi: 10.1109/MC.2012.37.
- [2] A. Ledmi, H. Bendjenna, y S. M. Hemam, “Fault Tolerance in Distributed Systems: A Survey,” en *Proc. 2018 3rd Int. Conf. Pattern Anal. Intell. Syst. (PAIS)*, Tebessa, Argelia, oct. 2018, pp. 1–5, doi: 10.1109/PAIS.2018.8598484.
- [3] D. A. Menascé, “MOM vs. RPC: Communication Models for Distributed Applications,” *IEEE Internet Comput.*, vol. 9, no. 2, pp. 90–93, mar./abr. 2005, doi: 10.1109/MIC.2005.42.
- [4] A. Lakshman y P. Malik, “Cassandra: A Decentralized Structured Storage System,” *ACM SIGOPS Oper. Syst. Rev.*, vol. 44, no. 2, pp. 35–40, abr. 2010, doi: 10.1145/1773912.1773922.
- [5] A. S. Tanenbaum y M. Van Steen, *Distributed Systems: Principles and Paradigms*, 2.^a ed. Upper Saddle River, NJ: Pearson Prentice Hall, 2007, ISBN: 978-0-13-239227-3.